

ROBOTICS - MINI PROJECT NUMBER 2

Arvin Mohammadi
student number: 810698303

November 6, 2022

Contents

0	INTRODUCTION	3
0.1	Part 1 - Mask Detection	3
0.2	Part 2 - Cube Detection	3
1	MASK DETECTION	4
1.1	Set Up	4
1.2	Face Detection	4
1.2.1	Haar Features	4
1.2.2	Python Implementation	6
1.3	Mask Detection	8
2	CUBE DETECTION	11
2.1	Set Up	11
2.2	Binary Matrix	12
2.3	Position	13
2.4	Orientation	14
3	APPENDIX A	18
3.1	Mask Detection	18
4	REFERENCES	21

0 INTRODUCTION

THIS is the second miniproject of the robotics course. The topic at hand is image processing and use of neural networks.

0.1 Part 1 - Mask Detection

Final desired result in this practice is to pinpoint face of a human from the frontal view and to determine if they have a mask on. There are four steps that need to be implemented to have this result. So the overview of what is going to be done is as followed:

- Loading the trained model
- Face detection using OpenCV
- Resizing image
- Determining if the detected face has a mask.

0.2 Part 2 - Cube Detection

Knowing the exact place of an object in a pick and place problem is one of the most important parts of the task. The steps for this, are as followed:

- Converting color coding
- Converting to binary matrix
- Detecting location and rotation

1 MASK DETECTION

PART 1 of the mini-project is dedicated to using OpenCV python package and a pre-trained neural network to detect the mask on people's faces.

1.1 Set Up

The first thing to do is setting up the virtual environment for the project and then installing the needed packages. To create a virtual environment, the following command is issued:

```
python -m venv robotics
.\robotics\scripts\activate
```

as for installing the dependencies:

```
pip install -U --force-reinstall package_name
```

The packages installed for this mini-project are: *numpy*, *cv2*, *tensorflow*, *jupyter* loading the pre-trained neural network model as as shown below:

```
from keras.models import load_model
mymodel = load_model('model.h5')
```

1.2 Face Detection

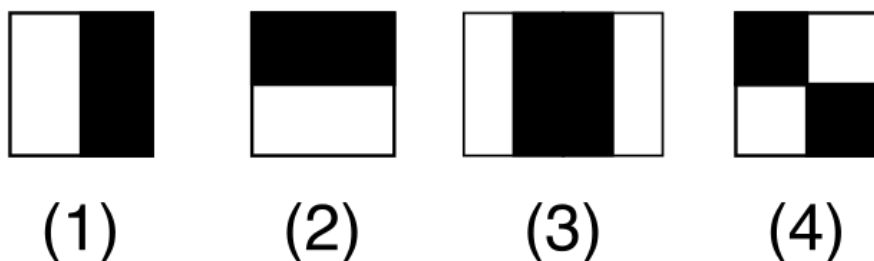
Here, the file *haarcascade_frontalface_default.xml* is used to detect the frontal view of the face using Haar features.

1.2.1 Haar Features

Face recognition means identifying the person. The key aspect in face recognition is detecting relevant features in human face like eyes, eyebrows, nose, lips. How does one detect these features in real-time? The answer is Haar features and the algorithm used, is called as Viola-Jones.

Haar features are sequence of rescaled square shaper functions proposed by Alfred Haar in 1909. They are similar to convolution kernels taught in the CNN courses. These haar features are applied to all relevant parts of face so as to detect human face. [Ref. 1]

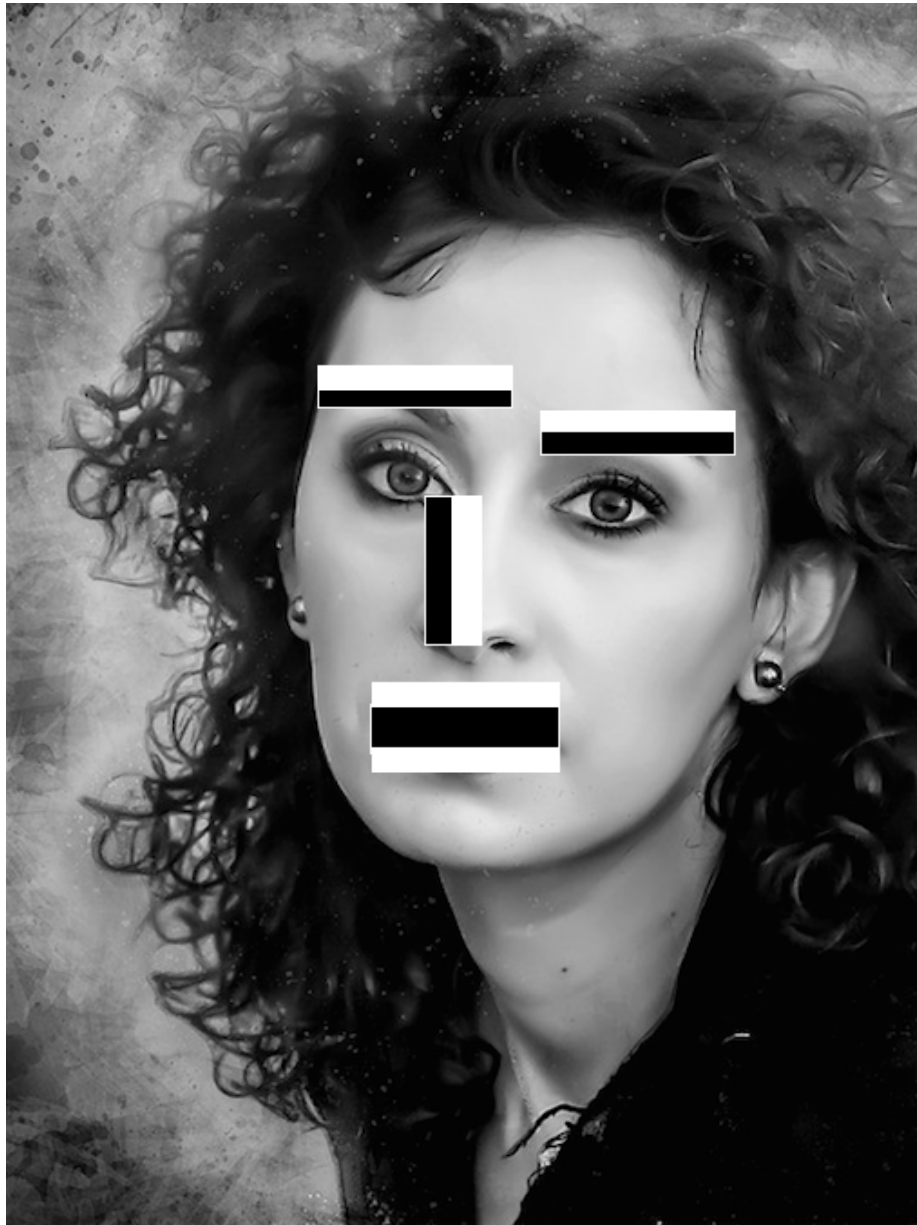
Figure 1: Haar features



Example of Haar Features: square shaped kernels

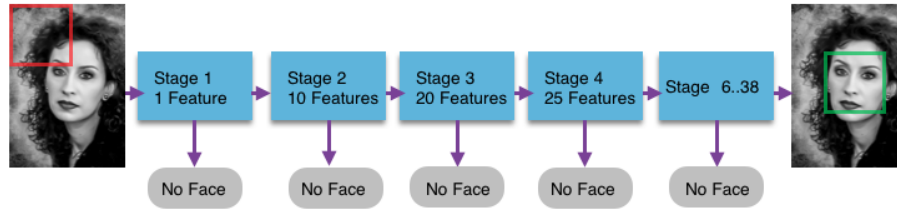
These features are used to find facial patterns.

Figure 2: Haar features used in facial recognition



Haar features applied on relevant parts of face

Figure 3: Face recognition



Haar cascade classifiers method

1.2.2 Python Implementation

The code for using the given file is as written below:

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades +
                                     'haarcascade_frontalface_default.xml')

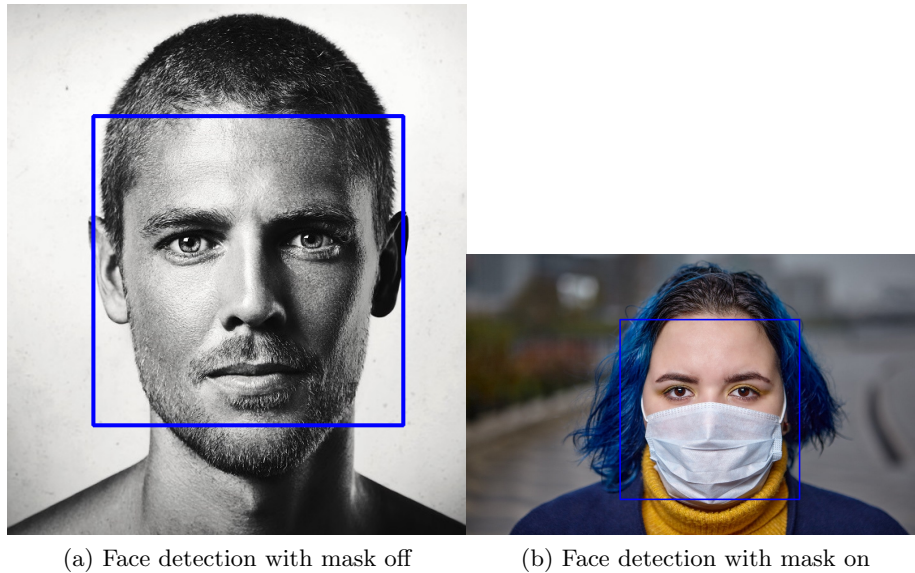
# read input image
img = cv2.imread('test (2).jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
faces = face_cascade.detectMultiScale(gray, 1.1, 4)

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x,y), (x+w, y+h), (255,0,0), 3)

# display the output
cv2.imshow('img', img)
cv2.waitKey()
```

The result for two samples are as followed:

Figure 4: Face detection using OpenCV

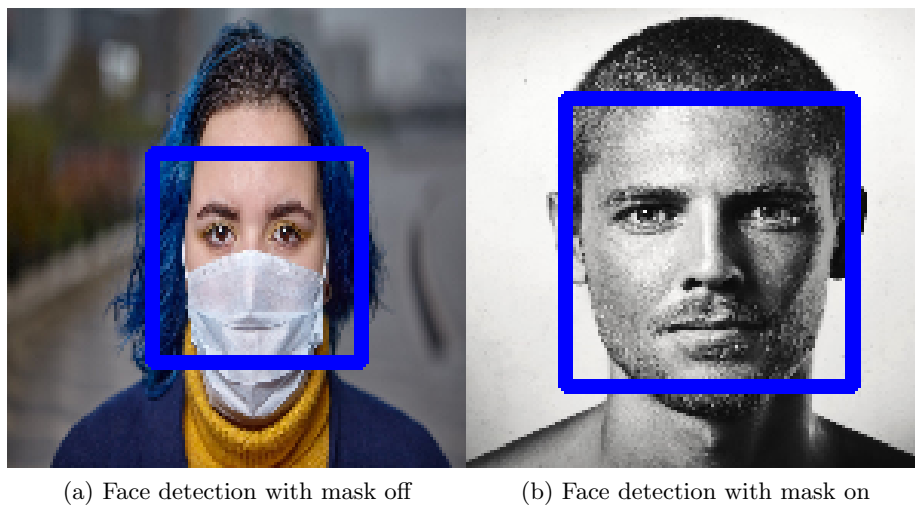


The algorithm can detect both faces, with or without the mask

Now the thing that needs to be done is resizing the images to 150×150 pixels. The code for this task is written as:

```
img = cv2.resize(img, (150, 150))
```

Figure 5: Face detection using OpenCV - resized



The algorithm can detect both faces, with or without the mask

1.3 Mask Detection

The last subsection is dedicated to mask detection, using the previous sections as a stepping-stone. The code for using the model on a sample image is:

```
img = np.resize(img, (1, 150, 150, 3))

prediction = mymodel.predict(img)

for i in prediction:
    if i == [1.]:
        print("unmasked")
    elif i == [0.]:
        print("masked")
```

which give the result:

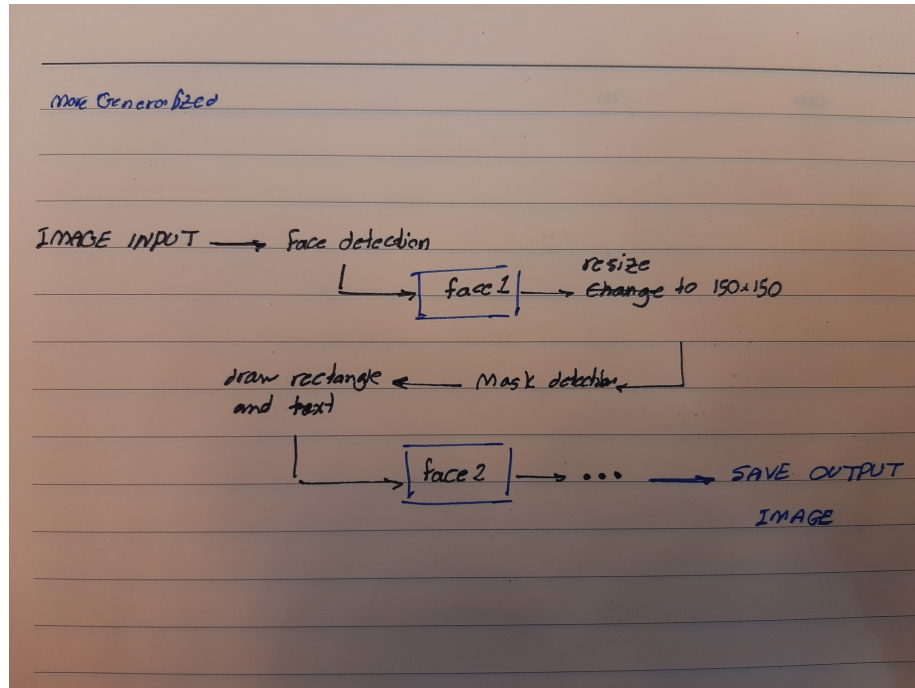
```
1/1 [=====] - 0s 66ms/step
masked
```

so now all that's needed to be done is constructing the code as a whole and evaluate the performance (precision) of the models. [Appendix A - 3.1]

after testing all the given data it seems that the model is not performing brilliantly. The algorithm used for mask detection is as followed:

1. Take image as input
2. Perform face detection
3. Loop over the faces detected inside the input image
 - (a) Crop the faces and resize them to 150×150 pixels
 - (b) Perform mask detection on the face
 - (c) Add the rectangle and label to the original image
4. save the output image (The image with added rectangles and images)

Figure 6: Algorithm



The algorithm used for mask detection

Despite having used the correct methods, the model performed poorly as shown by the examples below:

Figure 7: Mask detection



The algorithm does not perform well for determining the use of mask

As it stands, the face detection is being done with normal accuracy and precision (Around 60%), but the pre-trained NN does not detect the masks well. It mostly thinks that every face has a mask on.

2 CUBE DETECTION

KNOWING the exact location and orientation of an object is necessary for pick-and-place operations.

2.1 Set Up

Using the previously built virtual environment, a new jupyter notebook is run.

Then it's time to import the dependencies as:

```
import numpy as np
import cv2
import os
import math
```

adding image paths

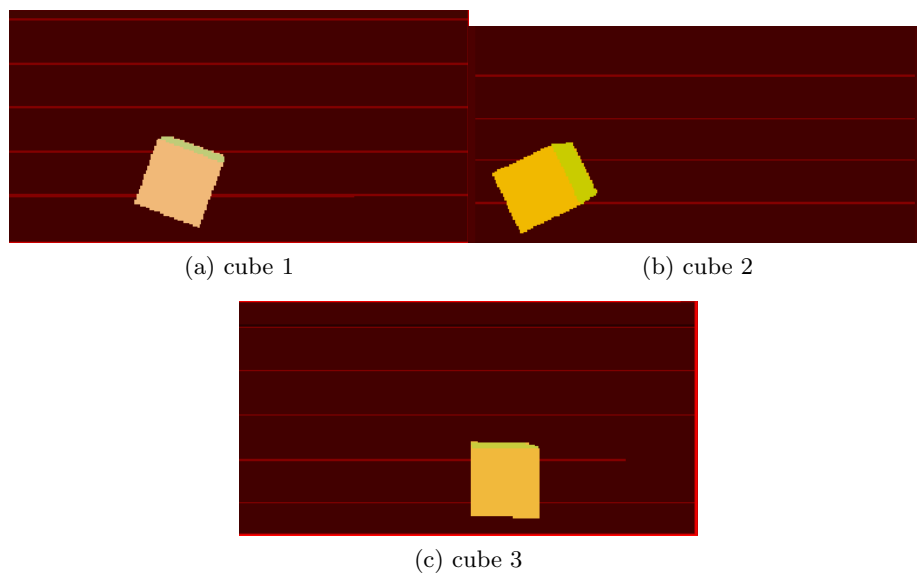
```
# intializing image_path list
images = []

# adding image paths to images list
image_path = 'Pictures/'
all_images = os.listdir(image_path)
for image in all_images:
    images.append(image_path + image)
images
```

output:

```
['Pictures/pic1.png', 'Pictures/pic2.png', 'Pictures/pic3.png']
```

Figure 8: Cubes - HSV



Color code change - results

2.2 Binary Matrix

The next part is to change the pictures to binary images. the code for this part is as followed:

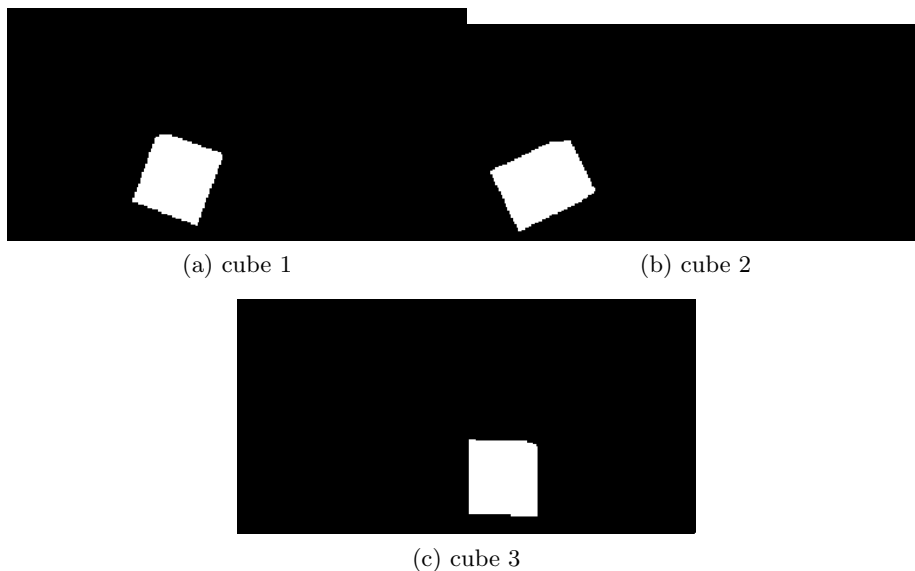
```
counter = 0
for image in images:
    counter += 1
    img = cv2.imread(image)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    cv2.imshow('image', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
    cv2.imwrite(image[:-4] + ' - hsv.png', img)

    lower_bound = np.array([0, 5, 0])
    upper_bound = np.array([255, 255, 255])

    imagemask = cv2.inRange(img, lower_bound, upper_bound)
    cv2.imshow('image', imagemask)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
    cv2.imwrite(image[:-4] + ' - new.png', imagemask)
```

and the results are:

Figure 9: Cubes - binary



Changing the colors to binary (0 and 1)

2.3 Position

For finding the location of the detected object first we have to draw a contour around the pixels with the value of one. After drawing the contour it is possible to calculate the centroid, using simple mathematics. The code for this task is given as:

```
counter = 0
for image in images:

    # keeping count of the images
    counter += 1

    # turning images to HSV
    img = cv2.imread(image)
    img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    # thresholding
    lower_bound = np.array([0, 5, 0])
    upper_bound = np.array([255, 255, 255])
    imagemask = cv2.inRange(img_hsv, lower_bound, upper_bound)
    cv2.imshow("image", imagemask)
    cv2.waitKey(0)
    cv2.destroyAllWindows

    # finding the contours
    cnts = cv2.findContours(imagemask.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnts = imutils.grab_contours(cnts)

    center_counter = 0
    for c in cnts:
        center_counter += 1

    if center_counter == 1 and counter == 3:
        continue

    M = cv2.moments(c)
    try:
        cX = int(M["m10"] / M["m00"])
        cY = int(M["m01"] / M["m00"])
    except:
        pass

    cv2.drawContours(img, [c], -1, (0, 0, 255), 2)
    cv2.circle(img, (cX, cY), 7, (255, 255, 255), -1)
    cv2.putText(img, "center", (cX-20, cY-20), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 255, 255),

    print(cX, cY)
    cv2.imshow("Image", img)
    cv2.waitKey(0)
```

```
cv2.destroyAllWindows
```

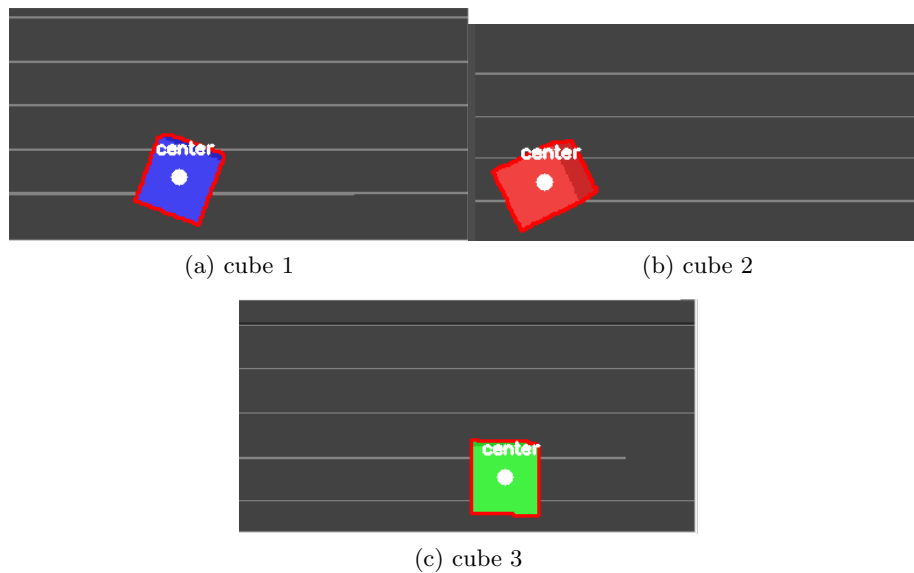
```
cv2.imwrite(image[: -4] + ' - centroid.png', img)
```

Output of location of the centroids are:

```
145 144  
63 130  
228 152
```

And finally, the resulting images are as followed:

Figure 10: Centroids



The script above, can calculate and show the centroids of the cubes

2.4 Orientation

In the last part of this mini-project the angle about z - *axis* is calculated. The steps to be taken for this job are explained as followed:

1. Read the image
2. Convert to HSV
3. Convert to binary using thresholding
4. Get outer contour
5. Get min Area Rectangle from the contour
6. Get vertices of rotated rectangle
7. Print the angle

8. save the image with the rotated rectangle drawn on it

The script:

```
counter = 0
for image in images:

    # keeping count of images
    counter += 1

    # turning images to HSV
    img = cv2.imread(image)
    img_hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

    # thresholding
    lower_bound = np.array([0, 5, 0])
    upper_bound = np.array([255, 255, 255])
    imagemask = cv2.inRange(img_hsv, lower_bound, upper_bound)

    # finding the contours
    cnts = cv2.findContours(imagemask.copy(), cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnts = imutils.grab_contours(cnts)

    # get rotated rectangle from outer contour
    rorect = cv2.minAreaRect(cnts[0])
    box = cv2.boxPoints(rorect)
    box = np.int0(box)

    # draw rotated rectangle on copy of img as result
    result = img.copy()
    cv2.drawContours(result, [box], 0, (0, 255, 0), 2)

    # get angle from rotated rectangle
    angle = rorect[-1]

    # from https://www.pyimagesearch.com/2017/02/20/text-skew-correction-opencv-python/
    # the 'cv2.minAreaRect' function returns values in the
    # range [-90, 0); as the rectangle rotates clockwise the
    # returned angle trends to 0 -- in this special case we
    # need to add 90 degrees to the angle
    if angle < -45:
        angle = -(90 + angle)

    # otherwise, just take the inverse of the angle to make
    # it positive
    else:
        angle = -angle

    print(angle, "deg")

    # write result to disk
```

```

cv2.imwrite("wing2_rotrect.png", result)

#      cv2.imshow("THRESH", imagemask)
cv2.imshow("RESULT", result)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imwrite(image[:-4] + ' - angle.png', result)

```

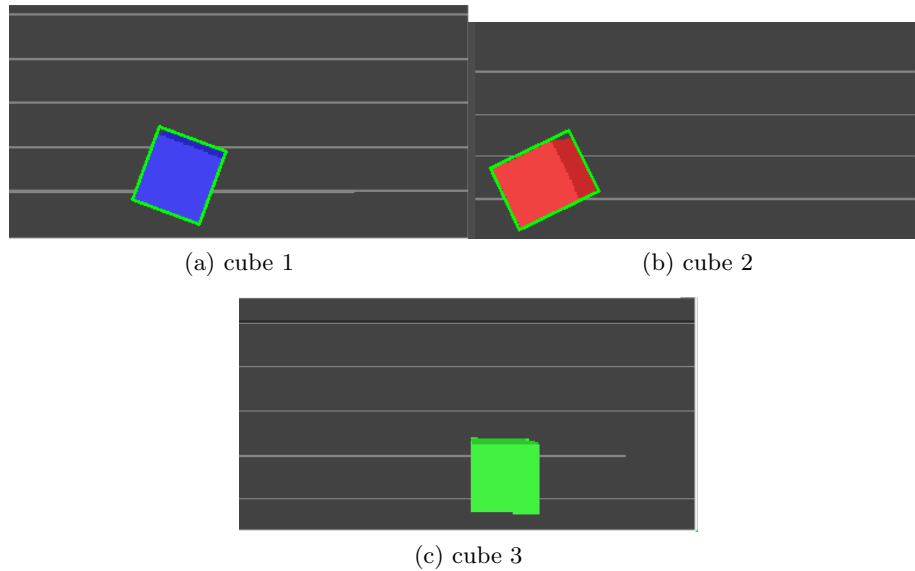
and the output angles are as followed:

```

-19.79887580871582 deg
-64.29004669189453 deg
-0.0 deg

```

Figure 11: Centroids



The script above, can calculate and show the angles of the cubes about $z - axis$

of course using the output angles, calculating the rotation matrix about $z - axis$ is beyond simple.

all we have to do is use the $rotz(angle)$ command and the rotations matrices in order are:

```

rotz(-19.80)
rotz(-64.29)
rotz(0.0)

```

output is:

$$Rot_z = \begin{bmatrix} 0.9409 & 0.3387 & 0 \\ -0.3387 & 0.9409 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$Rot_z = \begin{bmatrix} 0.4338 & 0.9010 & 0 \\ -0.9010 & 0.4338 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$Rot_z = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

3 APPENDIX A

A PPENDIX A is for the python codes and alorithms used in this mini-project.

3.1 Mask Detection

Mini project 2 - Image Processing (Part 1: Mask Detection)

Loading the pretrained model

```
from keras.models import load_model
from tensorflow.keras.applications.resnet50 import decode_predictions, preprocess_input

mymodel = load_model('model.h5')
```

```
mymodel.summary()
```

output

```
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d_7 (MaxPooling 2D)	(None, 74, 74, 32)	0
conv2d_8 (Conv2D)	(None, 72, 72, 32)	9248
max_pooling2d_8 (MaxPooling 2D)	(None, 36, 36, 32)	0
conv2d_9 (Conv2D)	(None, 34, 34, 32)	9248
max_pooling2d_9 (MaxPooling 2D)	(None, 17, 17, 32)	0
flatten_3 (Flatten)	(None, 9248)	0
dense_5 (Dense)	(None, 100)	924900
dense_6 (Dense)	(None, 1)	101
Total params: 944,393		
Trainable params: 944,393		
Non-trainable params: 0		

Face detection

imports

```

import cv2
import numpy as np
from PIL import Image, ImageFont, ImageDraw
import os
import time
from math import isclose

```

listing nth images path and labels

```

# loading the cascade file and haar features
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_defa

# assigning the path to the data
data_path = 'archive/'

# initializing h
images = []
labels = []

# adding the image and label to the list
dir_list = os.listdir(data_path)
for folder in dir_list:
    image_list = os.listdir(data_path + folder)
    for image in image_list:
        images.append(data_path + folder + "/" + image)
        labels.append(folder)

```

constructing the output image

```

for image_path in images:

    image_path_split = image_path.split('/')

    # == face detection =====
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, 1.1, 4)

    pil_image = Image.open(image_path)
    draw_image = ImageDraw.Draw(pil_image)

    # == mask detection looping over faces =====
    for (x, y, w, h) in faces:

        cropped_img = img[y:y+h, x:x+w]

        if image is None:
            continue

        cropped_img = cv2.resize(cropped_img, (150, 150))
        cv2.imwrite("temp.png", cropped_img)

```

```

# predictions
cropped_img = np.resize(cropped_img, (1, 150, 150, 3))
prediction = mymodel.predict(cropped_img)

# drawing border and text
shape = [(x, y), (x+w, y+h)]

if isclose(prediction[0, 0], 1):
    draw_image.rectangle(shape, outline="green")
    draw_image.text((x, y), "Un-masked", fill=(0, 255, 0))
elif isclose(prediction[0, 0], 0):
    draw_image.rectangle(shape, outline="red")
    draw_image.text((x, y), "Masked", fill=(255, 0, 0))

# saving final result
pil_image.save('results/' + image_path_split[-1])

```

4 REFERENCES

1. Online Article - Haar Features
2. Youtube video - Haar features, coding face detection